UNIVERSIDADE CATÓLICA DE SANTOS

Ciência da Computação

UV MAPPING DINÂMICO E PROCEDURAL

Computação Gráfica e Processamento de Imagens

Integrantes:

Arthur Almeida Lima - 3093901
Felipe Barbosa dos Santos - 4324223
João Gabriel Henriques Cardoso - 6100450
Lucas Carmona Neto - 8342055
Lucas Cerqueira Galvão - 4477944

Santos – SP
2026

Sumário

1 Introdução **5**

2 Fundamentação teórica **6**

2.1 UV Mapping tradicional 6

2.2 UV Mapping dinâmico 6

2.3 Mapeamento procedural 7

2.4 Vantagens 7

2.5 Limitações 8

2.6 Custo computacional 8

3 Equações e funcionamento matemático **9**

3.1 Coordenadas UV 9

3.2 Translação 9

3.3 Escala 10

3.4 Rotação 10

3.5 Transformação combinada 10

3.6 Interpolação 11

3.7 Mapeamento procedural 11

3.8 Relação com o pipeline 11

4 Relação com Vulkan **12**

4.1 Visão geral 12

4.2 Shaders 12
4.3 Buffers 13
4.4 Imagens, image views e samplers 13
4.5 Pipeline gráfico 13
4.6 Implementação dinâmica 13
4.7 Implementação procedural 14
4.8 Otimização 14

5 Arquitetura do Pipeline Vulkan 15

5.1 Visão geral da arquitetura 15
5.2 Fluxo de dados 16
5.3 Vertex Buffer e Index Buffer 16
5.4 Uniform Buffer 16
5.5 Descriptor Set e Bindings 17
5.6 Textura, Image View e Sampler 17
5.7 Vertex Shader 18
5.8 Rasterização e interpolação das coordenadas UV 18
5.9 Fragment Shader 18
5.10 Render Pass e Subpass 19
5.11 Pipeline Stages 19
5.12 Barreiras de memória 20
5.13 Transições de Image Layout 21
5.14 Dependências de execução 21
5.15 Relação com o UV Mapping Dinâmico e Procedural 22
5.16 Justificativa das decisões arquiteturais 22

6 Pseudocódigo dos Shaders 24

6.1 Objetivo dos shaders 24

6.2 Vertex Shader 24
6.2.1 Transformação da posição 25
6.2.2 Escala das coordenadas UV 25
6.2.3 Deslocamento temporal 26
6.3 Pseudocódigo do Vertex Shader 26
6.4 Rasterização e interpolação 27
6.5 Fragment Shader 28
6.6 Amostragem da textura 28
6.7 Função procedural 29
6.8 Normalização e iluminação 29
6.9 Combinação dos resultados 30
6.10 Tratamento de limites 30
6.11 Pseudocódigo do Fragment Shader 31
6.12 Fluxo completo dos dados 33
6.13 Relação com o UV Mapping Dinâmico e Procedural 34

7 Aplicações práticas **36**
7.1 Superfícies de água e materiais móveis 36
7.2 Terrenos e ambientes virtuais 36
7.3 Materiais procedurais 36
7.4 Efeitos visuais 37

8 Utilização em jogos **38**
8.1 Rocket League 38
8.2 Marvel's Spider-Man Remastered 38
8.3 Comparação 39

9 Capturas de gameplay **40**

9.1 Capturas de Gameplay 40
9.2 Rocket League 40
9.3 Marvel's Spider-Man Remastered 41
9.4 Limitação das Capturas 43

10 Vídeos de demonstração **44**
10.1 Vídeos de demonstração 44
10.2 Rocket League 44
10.3 Marvel's Spider-Man Remastered 44

11 Resumo das publicações utilizadas **46**
11.1 Publicação 1 – Heckbert (1986) 46
11.2 Publicação 2 – Dong et al. (2020) 46
11.3 Publicação 3 – Smelik et al. (2014) 47
11.4 Relação entre as publicações 47

12 Conclusão **48**

A Checklist da entrega **51**

Capítulo 1

Introdução

O mapeamento de texturas, conhecido como *texture mapping*, é uma técnica fundamental da Computação Gráfica para associar informações bidimensionais à superfície de objetos tridimensionais. Em sua forma tradicional, coordenadas denominadas U e V estabelecem a correspondência entre pontos de uma superfície 3D e posições em uma textura 2D. Heckbert [1] apresenta o mapeamento de texturas como uma técnica capaz de aumentar significativamente a riqueza visual das imagens sem exigir que todos os detalhes da superfície sejam modelados geometricamente.

A partir dessa base, o mapeamento pode ser modificado durante a execução da aplicação. No caso do UV mapping dinâmico, as coordenadas podem sofrer translação, escala, rotação ou outras transformações dependentes de tempo, posição ou parâmetros de animação. Já o mapeamento procedural utiliza funções matemáticas e outros procedimentos para gerar padrões ou coordenadas sem depender exclusivamente de uma textura previamente desenhada.

Este trabalho apresenta uma revisão bibliográfica sobre UV mapping dinâmico e procedural, abordando seus fundamentos, equações, funcionamento no pipeline gráfico, relação com a API Vulkan, aplicações práticas e utilização em jogos. Também são analisadas duas aplicações em jogos: *Rocket League* e *Marvel's Spider-Man Remastered*.

A revisão utiliza, entre outras fontes, o trabalho clássico de Heckbert sobre texture mapping, o levantamento de Dong et al. sobre geração procedural de texturas bidimensionais e o levantamento de Smelik et al. sobre modelagem procedural para mundos virtuais. Os resumos dessas publicações são apresentados em capítulo próprio.

Capítulo 2

Fundamentação teórica

2.1 UV Mapping tradicional

O UV mapping estabelece uma correspondência entre a superfície de um objeto tridimensional e uma imagem bidimensional. As coordenadas U e V normalmente são normalizadas no intervalo $[0, 1]$. Dessa maneira, um vértice do modelo pode carregar, além de sua posição e outros atributos, uma coordenada que indica qual região da textura deve ser associada àquele ponto.

No pipeline de renderização, as coordenadas dos vértices são fornecidas ao estágio de vértice e, durante a rasterização, são interpoladas para os fragmentos. O fragment shader pode então utilizar as coordenadas interpoladas para consultar uma textura por meio de um sampler.

Heckbert [1] destaca dois aspectos centrais do texture mapping: o mapeamento geométrico, responsável por relacionar a textura à superfície, e a filtragem, necessária para reduzir artefatos como aliasing.

2.2 UV Mapping dinâmico

O UV mapping dinâmico consiste em modificar as coordenadas de textura durante a execução. Uma transformação simples é a translação:

$$u' = u + \Delta u, \qquad v' = v + \Delta v.$$

Se Δu ou Δv variarem com o tempo, a textura pode apresentar movimento contínuo.

Esse mecanismo pode ser empregado para representar, por exemplo, o deslocamento visual de água, energia, lava ou elementos de interface.

Também podem ser aplicadas escala e rotação. A combinação dessas operações permite controlar a velocidade, direção, repetição e orientação de uma textura sem modificar a malha original.

2.3 Mapeamento procedural

No mapeamento procedural, o resultado é produzido por uma função ou procedimento. Dong et al. [2] definem a geração procedural de texturas como o processo de criação de texturas por meio de modelos matemáticos e apresentam uma revisão de diferentes métodos para geração de texturas bidimensionais.

Uma formulação simplificada pode ser escrita como:

$$C = f(P, t),$$

em que P representa uma posição espacial, t representa uma variável temporal quando necessário e f representa a função procedural responsável pelo resultado.

A abordagem procedural é útil quando se deseja gerar padrões repetitivos, variações ou materiais sem armazenar uma imagem diferente para cada situação. Ela também pode ser combinada com UV coordinates tradicionais.

2.4 Vantagens

Entre as principais vantagens estão a flexibilidade, a possibilidade de animação em tempo real, o reaproveitamento de texturas e a geração de variações sem necessidade de armazenar todos os resultados previamente.

Métodos procedurais também podem reduzir a quantidade de dados necessários para representar determinados tipos de conteúdo. Smelik et al. [3] destacam que métodos procedurais são atrativos para a geração de grandes quantidades de conteúdo em ambientes virtuais, embora o controle artístico e a interatividade continuem sendo desafios importantes.

2.5 Limitações

A técnica não elimina os problemas tradicionais de mapeamento. Podem ocorrer distorções, descontinuidades, repetição excessiva de padrões e artefatos de filtragem. No caso procedural, funções muito complexas também podem aumentar o custo computacional.

Outro ponto é o controle artístico: uma função procedural pode gerar grande variedade, mas é necessário definir parâmetros que permitam ao artista controlar o resultado desejado.

2.6 Custo computacional

O custo depende principalmente da quantidade de cálculos executados por fragmento, do número de amostragens de textura, dos filtros utilizados e da resolução envolvida. Operações realizadas no fragment shader podem ser executadas muitas vezes, pois um mesmo shader pode processar grande quantidade de fragmentos.

Por isso, uma implementação eficiente deve avaliar quais cálculos realmente precisam ser feitos por fragmento e quais podem ser antecipados ou simplificados.

Capítulo 3

Equações e funcionamento matemático

3.1 Coordenadas UV

As coordenadas básicas são representadas por:

$$(u, v), \qquad 0 \leq u \leq 1, \quad 0 \leq v \leq 1. \tag{3.1}$$

u representa a posição horizontal da textura e v representa a posição vertical. Os valores normalizados permitem acessar a textura independentemente de sua resolução.

3.2 Translação

A translação das coordenadas é dada por:

$$u' = u + \Delta u, \qquad v' = v + \Delta v. \tag{3.2}$$

Os termos Δu e Δv determinam o deslocamento. Se o deslocamento for atualizado com o tempo, a textura pode aparentar movimento.

Uma forma simples é:

$$u' = u + s_u t, \qquad v' = v + s_v t, \tag{3.3}$$

em que s_u e s_v representam velocidades de deslocamento.

3.3 Escala

A escala pode ser definida por:

$$u' = s_u u, \qquad v' = s_v v. \tag{3.4}$$

Valores maiores que 1 aumentam a frequência aparente da textura, enquanto valores menores reduzem sua frequência.

3.4 Rotação

Considerando uma rotação em torno da origem do espaço UV:

$$u' = u\cos\theta - v\sin\theta, \tag{3.5}$$

$$v' = u\sin\theta + v\cos\theta. \tag{3.6}$$

O parâmetro θ representa o ângulo de rotação.

3.5 Transformação combinada

As operações podem ser representadas matricialmente:

$$P' = MP, \tag{3.7}$$

onde $P = (u, v)^T$ é o ponto original e M representa a transformação escolhida.

3.6 Interpolação

Para um triângulo, as coordenadas UV dos fragmentos podem ser obtidas por interpolação baricêntrica:

$$UV = \alpha UV_0 + \beta UV_1 + \gamma UV_2, \tag{3.8}$$

com:

$$\alpha + \beta + \gamma = 1. \tag{3.9}$$

Assim, os atributos dos vértices são utilizados para obter valores correspondentes em cada fragmento.

3.7 Mapeamento procedural

Uma formulação geral é:

$$UV' = f(P, t), \tag{3.10}$$

ou, quando o objetivo é produzir diretamente a cor:

$$C = f(P, t). \tag{3.11}$$

Essas expressões representam a ideia de que o resultado pode ser calculado a partir da posição espacial e, opcionalmente, do tempo.

3.8 Relação com o pipeline

O fluxo conceitual pode ser resumido como:

$$\textit{Coordenadas UV} \rightarrow \textit{Transformação} \rightarrow \textit{Interpolação}$$
$$\rightarrow \textit{Fragment Shader} \rightarrow \textit{Sampling} \rightarrow \textit{Cor final}$$

Capítulo 4

Relação com Vulkan

4.1 Visão geral

Vulkan é uma API gráfica de baixo nível que fornece maior controle explícito sobre recursos e etapas da renderização. Para o mapeamento de texturas, são especialmente importantes os buffers, imagens, image views, samplers, shaders e o pipeline gráfico.

4.2 Shaders

As coordenadas UV podem ser armazenadas como atributos dos vértices. O vertex shader recebe esses atributos e os encaminha para o estágio seguinte. Após a rasterização, as coordenadas são interpoladas e disponibilizadas ao fragment shader.

Uma transformação dinâmica simples pode ser implementada conceitualmente como:

$$u' = u + v_t t, \tag{4.1}$$

onde v_t representa uma velocidade e t o tempo.

Para uma aplicação procedural, o shader pode calcular um valor a partir das coordenadas e de outras entradas:

$$C = f(UV, P, t). \tag{4.2}$$

4.3 Buffers

Um vertex buffer pode armazenar posição, normal e coordenadas UV. Dados que mudam durante a execução, como tempo, velocidade de animação ou escala, podem ser enviados por meio de buffers de uniformes.

4.4 Imagens, image views e samplers

No Vulkan, a imagem de textura é um recurso distinto de sua image view e do sampler. A documentação do Vulkan explica que shaders normalmente acessam texturas por meio de samplers, que aplicam operações de filtragem durante a leitura dos texels [5].

O sampler também participa de decisões como filtragem e comportamento de endereçamento. Esses mecanismos são importantes para evitar resultados visuais inadequados quando uma textura é ampliada ou reduzida.

4.5 Pipeline gráfico

O fluxo simplificado pode ser representado por:

$$\textit{Vertex Buffer} \rightarrow \textit{Vertex Shader} \rightarrow \textit{Rasterização}$$
$$\rightarrow \textit{Interpolação UV} \rightarrow \textit{Fragment Shader} \rightarrow \textit{Sampler + Image}$$
$$\rightarrow \textit{Framebuffer/Attachment}$$

No caso do UV mapping dinâmico, a transformação pode ser realizada antes da consulta à textura. No caso procedural, o shader pode calcular diretamente os valores necessários para produzir o padrão ou modificar as coordenadas.

4.6 Implementação dinâmica

Uma implementação dinâmica pode utilizar um parâmetro de tempo atualizado pela aplicação. O shader recebe esse parâmetro e aplica a transformação às coordenadas UV. Dessa forma, a mesma textura pode ser reutilizada com diferentes velocidades, direções ou escalas.

4.7 Implementação procedural

Na abordagem procedural, a aplicação pode fornecer parâmetros como tempo, posição e escala, enquanto o shader calcula o padrão. Uma função periódica, por exemplo, pode produzir variações que se deslocam ao longo da superfície.

4.8 Otimização

A otimização deve considerar que cálculos no fragment shader podem ser executados para grande quantidade de fragmentos. Operações desnecessárias devem ser reduzidas, e os parâmetros devem ser organizados para evitar transferências excessivas.

Também é importante utilizar adequadamente filtragem e mipmapping quando o conteúdo for armazenado em texturas, pois a escolha do método de amostragem influencia a qualidade e o custo da renderização.

Capítulo 5

Arquitetura do Pipeline Vulkan

5.1 Visão geral da arquitetura

A arquitetura apresentada nesta etapa representa o fluxo de dados necessário para utilizar o Vulkan na implementação de UV Mapping Dinâmico e Procedural. O objetivo é relacionar os recursos da API com as etapas de processamento responsáveis pela transformação e utilização das coordenadas de textura.

A Figura 5.1 apresenta uma visão geral da arquitetura. Os recursos em azul representam dados e recursos utilizados pela aplicação, os elementos em verde representam etapas de processamento e shaders, os elementos em laranja representam mecanismos de sincronização e os elementos em roxo representam o Render Pass e seus attachments. Os elementos em amarelo destacam especificamente as operações relacionadas ao UV Mapping.

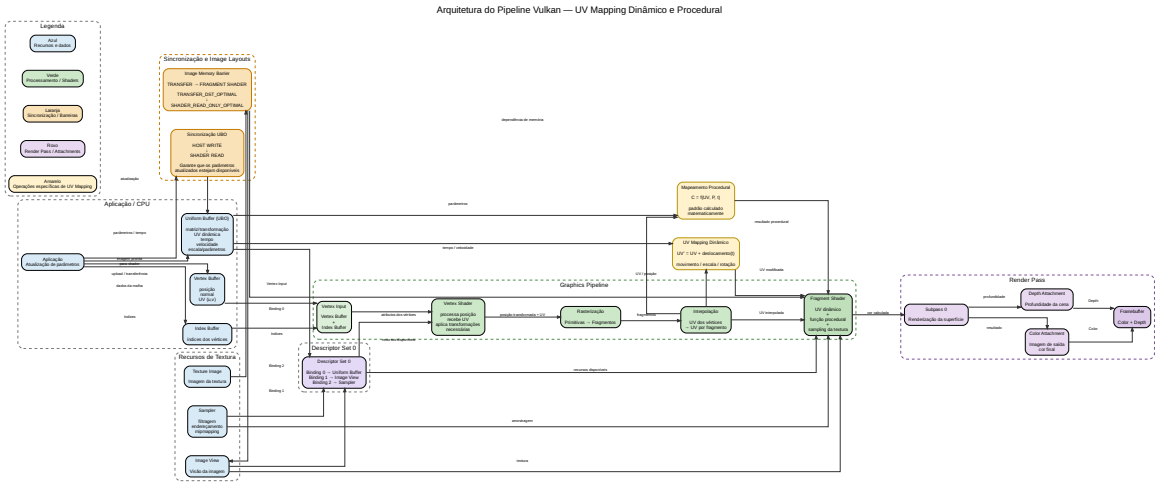


Figura 5.1: Arquitetura do pipeline Vulkan aplicada ao UV Mapping Dinâmico e Procedural.

5.2 Fluxo de dados

O processamento começa na aplicação, responsável por fornecer os dados da malha e os parâmetros necessários para a renderização. O Vertex Buffer armazena os atributos dos vértices, incluindo posição, normal e coordenadas UV. O Index Buffer armazena os índices utilizados para formar as primitivas da malha.

Além desses dados, a aplicação pode atualizar parâmetros durante a execução, como tempo, velocidade de deslocamento, escala e outros parâmetros utilizados pela técnica. Esses valores são armazenados em um Uniform Buffer (UBO) e disponibilizados aos shaders por meio do Descriptor Set.

O fluxo principal pode ser resumido como:

$$\textit{Vertex Buffer + Index Buffer} \rightarrow \textit{Vertex Input} \rightarrow \textit{Vertex Shader}$$
$$\rightarrow \textit{Rasterização} \rightarrow \textit{Interpolação UV} \rightarrow \textit{Fragment Shader}$$
$$\rightarrow \textit{Color Attachment} \rightarrow \textit{Framebuffer}$$

Durante esse processo, as coordenadas UV são transportadas pelos estágios do pipeline e podem ser modificadas para produzir o efeito dinâmico ou procedural desejado.

5.3 Vertex Buffer e Index Buffer

O Vertex Buffer contém os dados associados aos vértices da superfície renderizada. Para esta técnica, são relevantes principalmente a posição do vértice e suas coordenadas UV. As coordenadas UV indicam a posição correspondente na textura bidimensional.

O Index Buffer contém os índices utilizados para definir a ordem dos vértices que formam as primitivas. Dessa maneira, a geometria pode ser descrita sem a necessidade de repetir todos os atributos dos vértices.

Esses buffers são utilizados no estágio de entrada do pipeline e fornecem os dados que serão processados pelo Vertex Shader.

5.4 Uniform Buffer

O Uniform Buffer é utilizado para armazenar valores que podem ser atualizados pela aplicação e utilizados pelos shaders. No contexto do UV Mapping Dinâmico e Procedural,

podem ser armazenados parâmetros como tempo, velocidade de deslocamento, escala e parâmetros utilizados pelas funções procedurais.

Um exemplo é o deslocamento dependente do tempo:

$$u' = u + v_u t, \qquad v' = v + v_v t. \tag{5.1}$$

Nesse caso, v_u e v_v representam velocidades de deslocamento e t representa o tempo. A aplicação atualiza esses valores e o shader os utiliza durante a renderização.

5.5 Descriptor Set e Bindings

O Descriptor Set 0 organiza os recursos utilizados pelos shaders. Na arquitetura apresentada, são considerados três bindings principais.

Tabela 5.1: Organização do Descriptor Set 0.

Binding	**Recurso**	**Finalidade**
0	Uniform Buffer	Parâmetros dinâmicos e procedurais
1	Image View	Acesso à imagem da textura
2	Sampler	Filtragem e endereçamento

Essa organização permite que os shaders acessem os recursos necessários sem que eles sejam diretamente incorporados ao código do shader. O Uniform Buffer fornece os parâmetros variáveis, enquanto a Image View e o Sampler permitem realizar a leitura da textura.

5.6 Textura, Image View e Sampler

A textura utilizada durante a renderização é armazenada como uma Image resource. A Image View define como essa imagem será interpretada e acessada pelo shader, enquanto o Sampler define características da leitura, como filtragem e comportamento de endereçamento.

No fluxo apresentado, a imagem deve estar em um layout adequado para ser consultada pelo Fragment Shader. Para uma imagem utilizada como textura, o layout representado é:

`SHADER_READ_ONLY_OPTIMAL`

A separação entre Image, Image View e Sampler permite organizar o recurso de textura de acordo com as responsabilidades de cada objeto.

5.7 Vertex Shader

O Vertex Shader recebe os atributos fornecidos pelo Vertex Buffer. Entre esses atributos estão as coordenadas UV associadas aos vértices.

Nesse estágio, podem ser realizadas transformações necessárias antes da rasterização. A posição dos vértices também é processada para determinar sua posição no espaço utilizado pelo pipeline.

As coordenadas UV resultantes são encaminhadas para os estágios seguintes, onde serão interpoladas para os fragmentos.

5.8 Rasterização e interpolação das coordenadas UV

Depois do processamento dos vértices, as primitivas são submetidas à rasterização. Esse processo determina os fragmentos que correspondem às regiões da superfície projetada.

As coordenadas UV associadas aos vértices são interpoladas para cada fragmento. Para um triângulo, uma representação simplificada dessa interpolação é:

$$UV = \alpha UV_0 + \beta UV_1 + \gamma UV_2, \tag{5.2}$$

com:

$$\alpha + \beta + \gamma = 1. \tag{5.3}$$

Dessa forma, o Fragment Shader recebe coordenadas UV correspondentes a cada fragmento produzido pela rasterização.

5.9 Fragment Shader

O Fragment Shader é a etapa central para a aplicação dos efeitos de UV Mapping Dinâmico e Procedural.

No caso dinâmico, as coordenadas podem ser modificadas utilizando parâmetros fornecidos pelo Uniform Buffer. Por exemplo:

$$UV' = UV + St, \tag{5.4}$$

em que S representa uma velocidade de deslocamento e t representa o tempo.

No caso procedural, o shader pode calcular o resultado utilizando as coordenadas UV, a posição espacial e parâmetros temporais:

$$C = f(UV, P, t). \tag{5.5}$$

Assim, o shader pode utilizar uma textura existente, modificar suas coordenadas de acesso ou produzir diretamente valores relacionados ao padrão procedural.

5.10 Render Pass e Subpass

O Render Pass representa a organização das operações de renderização e dos attachments utilizados como destino dos resultados.

Na arquitetura proposta é utilizado um Subpass 0 responsável pela renderização da superfície. Esse Subpass recebe os resultados produzidos pelo Fragment Shader e os direciona para os attachments correspondentes.

O Color Attachment armazena a cor final produzida pela renderização. O Depth Attachment armazena informações de profundidade utilizadas para determinar a visibilidade das superfícies.

Os attachments são associados ao Framebuffer, que representa as imagens concretas utilizadas durante a execução do Render Pass.

5.11 Pipeline Stages

Os principais estágios envolvidos na arquitetura são:

- **Vertex Input:** recebe os dados dos Vertex e Index Buffers;
- **Vertex Shader:** processa os vértices e seus atributos;

- **Rasterização:** converte as primitivas em fragmentos;
- **Interpolação:** determina os atributos correspondentes aos fragmentos, incluindo as coordenadas UV;
- **Fragment Shader:** calcula a cor final e aplica as operações dinâmicas ou procedurais;
- **Color Attachment:** recebe o resultado da renderização.

Essa sequência relaciona os dados armazenados nos buffers aos resultados produzidos pelo Render Pass.

5.12 Barreiras de memória

As barreiras são utilizadas para estabelecer dependências entre acessos aos recursos. Na arquitetura apresentada, uma das situações relevantes ocorre quando uma imagem é inicialmente preenchida por uma operação de transferência e posteriormente utilizada pelo Fragment Shader.

A transição pode ser representada como:

$$\texttt{TRANSFER_DST_OPTIMAL} \rightarrow \texttt{SHADER_READ_ONLY_OPTIMAL}$$

A Image Memory Barrier garante que as operações de escrita realizadas durante a transferência sejam concluídas e que os dados estejam disponíveis antes da leitura realizada pelo shader.

A barreira representada no diagrama possui, conceitualmente, as seguintes características:

Tabela 5.2: Barreira de memória associada à textura.

Item	**Descrição**
Recurso	Texture Image
Acesso anterior	Escrita por transferência
Etapa anterior	Transferência
Acesso posterior	Leitura pelo shader
Etapa posterior	Fragment Shader
Layout anterior	`TRANSFER_DST_OPTIMAL`
Layout posterior	`SHADER_READ_ONLY_OPTIMAL`
Motivo	Garantir que a textura esteja disponível e corretamente posicionada antes da leitura pelo shader.

Também é necessária uma dependência para os dados atualizados no Uniform Buffer. A aplicação realiza a escrita dos parâmetros e o shader realiza a leitura desses dados durante a renderização. A sincronização garante que os valores atualizados estejam disponíveis antes do acesso pelo shader.

5.13 Transições de Image Layout

As imagens utilizadas pelo Vulkan possuem layouts que indicam como os recursos serão utilizados. No caso da textura, a imagem pode inicialmente estar em um estado inadequado para leitura pelo shader.

Após a operação de transferência, a imagem é colocada em:

`SHADER_READ_ONLY_OPTIMAL`

Esse layout representa a utilização da imagem como recurso de leitura pelo shader.

O diagrama representa essa dependência juntamente com a Image Memory Barrier, relacionando a mudança de layout à disponibilidade dos dados.

5.14 Dependências de execução

O fluxo possui dependências entre a produção e o consumo dos recursos. Os dados da malha precisam estar disponíveis antes da execução dos shaders, os parâmetros do Uniform

Buffer precisam estar atualizados antes de serem lidos e a textura precisa estar preparada antes do sampling.

A dependência mais importante relacionada à textura é:

$$\textit{Transferência da textura} \rightarrow \textit{Image Memory Barrier}$$
$$\rightarrow \textit{SHADER_READ_ONLY_OPTIMAL} \rightarrow \textit{Fragment Shader}$$

Essas dependências evitam que uma etapa consuma dados antes que a etapa anterior tenha terminado de produzi-los ou disponibilizá-los.

5.15 Relação com o UV Mapping Dinâmico e Procedural

A arquitetura apresentada está diretamente relacionada ao funcionamento da técnica estudada.

No UV Mapping Dinâmico, as coordenadas de textura são modificadas durante a execução utilizando parâmetros como tempo, velocidade, escala ou rotação. Esses parâmetros podem ser atualizados pela aplicação e disponibilizados aos shaders por meio do Uniform Buffer.

No mapeamento procedural, o Fragment Shader pode utilizar as coordenadas interpoladas, a posição espacial e parâmetros fornecidos pela aplicação para calcular um padrão ou uma cor. Uma formulação simplificada é:

$$C = f(UV, P, t). \tag{5.6}$$

Dessa maneira, a arquitetura permite reutilizar os mesmos recursos de geometria e textura enquanto o resultado visual é alterado durante a execução.

5.16 Justificativa das decisões arquiteturais

A utilização de Vertex Buffer e Index Buffer mantém os dados geométricos separados dos parâmetros que mudam durante a execução. O Uniform Buffer é utilizado para os valores dinâmicos porque esses dados podem ser atualizados sem alterar a estrutura da malha.

A organização dos recursos por meio de Descriptor Sets permite que os shaders acessem o UBO, a textura e o Sampler de maneira organizada. A separação entre Image, Image View e Sampler também corresponde à organização dos recursos de textura utilizada pelo Vulkan.

A aplicação das operações de UV Mapping nos shaders permite modificar o resultado sem alterar a geometria original. Essa característica é importante para efeitos dinâmicos e para métodos procedurais, nos quais os valores podem ser calculados durante a renderização.

Por fim, as barreiras e transições de layout representadas no diagrama estabelecem as dependências necessárias para que os recursos sejam corretamente produzidos e consumidos pelas diferentes etapas do pipeline.

Capítulo 6

Pseudocódigo dos Shaders

6.1 Objetivo dos shaders

Os shaders apresentados nesta etapa descrevem a lógica necessária para implementar o efeito de UV Mapping Dinâmico e Procedural em uma arquitetura gráfica baseada em Vulkan. O pseudocódigo utiliza uma sintaxe semelhante à linguagem GLSL, porém não tem como objetivo ser compilado diretamente.

Foram utilizados dois shaders principais: um Vertex Shader, responsável pelo processamento dos vértices e pela preparação das coordenadas UV, e um Fragment Shader, responsável pela aplicação do deslocamento dinâmico, pela amostragem da textura, pelo cálculo procedural e pela produção da cor final.

O fluxo geral pode ser representado por:

$$\textit{Vertex Buffer} \rightarrow \textit{Vertex Shader}$$
$$\rightarrow \textit{Rasterização e Interpolação} \rightarrow \textit{Fragment Shader}$$
$$\rightarrow \textit{Color Attachment} \rightarrow \textit{Framebuffer}$$

6.2 Vertex Shader

O Vertex Shader recebe os atributos associados aos vértices da malha. São utilizados a posição, a normal e as coordenadas UV. O shader também recebe por meio de um Uniform Buffer os parâmetros necessários para a transformação geométrica e para o comportamento dinâmico das coordenadas de textura.

Os principais dados utilizados são:

- **inPosition**: posição do vértice no espaço do modelo;
- **inNormal**: vetor normal associado ao vértice;
- **inUV**: coordenadas de textura do vértice;
- **model**: matriz de transformação do espaço do modelo para o espaço de mundo;
- **view**: matriz de transformação para o espaço de visualização;
- **projection**: matriz de projeção;
- **time**: tempo utilizado para produzir o movimento da textura;
- **uvVelocity**: velocidade de deslocamento das coordenadas UV;
- **uvScale**: escala aplicada às coordenadas UV.

6.2.1 Transformação da posição

A posição do vértice é transformada pelas matrizes Model, View e Projection. De forma simplificada:

$$P_{world} = M_{model} P_{local}, \tag{6.1}$$

$$P_{view} = M_{view} P_{world}, \tag{6.2}$$

$$P_{clip} = M_{projection} P_{view}. \tag{6.3}$$

O resultado é utilizado para determinar a posição do vértice na tela.

6.2.2 Escala das coordenadas UV

A escala permite controlar a repetição da textura na superfície. A operação é representada por:

$$UV_{scale} = UV \cdot S, \tag{6.4}$$

em que UV representa a coordenada original e S representa o fator de escala.

6.2.3 Deslocamento temporal

O deslocamento dinâmico utiliza o tempo de execução e uma velocidade definida pela aplicação:

$$UV_{offset} = Vt, \tag{6.5}$$

em que V representa a velocidade de deslocamento e t representa o tempo.

A coordenada enviada para o próximo estágio é então:

$$UV' = UV_{scale} + UV_{offset}. \tag{6.6}$$

Essa operação permite produzir movimento contínuo da textura sem modificar a geometria da malha.

6.3 Pseudocódigo do Vertex Shader

O arquivo `uv_mapping.vert` contém o pseudocódigo correspondente ao Vertex Shader:

```
layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inNormal;
layout(location = 2) in vec2 inUV;

layout(set = 0, binding = 0) uniform Parameters
{
    mat4 model;
    mat4 view;
    mat4 projection;

    float time;
    vec2 uvVelocity;
    vec2 uvScale;
} params;

layout(location = 0) out vec2 fragUV;
layout(location = 1) out vec3 fragNormal;
```

```
layout(location = 2) out vec3 fragPosition;

void main()
{
    vec4 worldPosition =
        params.model * vec4(inPosition, 1.0);

    vec4 viewPosition =
        params.view * worldPosition;

    gl_Position =
        params.projection * viewPosition;

    fragPosition = worldPosition.xyz;
    fragNormal = inNormal;

    vec2 scaledUV =
        inUV * params.uvScale;

    vec2 uvOffset =
        params.uvVelocity * params.time;

    fragUV =
        scaledUV + uvOffset;
}
```

6.4 Rasterização e interpolação

Após a execução do Vertex Shader, as primitivas são submetidas à rasterização. Durante esse processo, os atributos enviados pelos vértices são interpolados para os fragmentos.

Para um triângulo, a interpolação das coordenadas UV pode ser representada por:

$$UV = \alpha UV_0 + \beta UV_1 + \gamma UV_2, \tag{6.7}$$

com:

$$\alpha + \beta + \gamma = 1. \tag{6.8}$$

O resultado dessa interpolação é recebido pelo Fragment Shader como `fragUV`.

6.5 Fragment Shader

O Fragment Shader é responsável pela etapa principal do efeito visual. Ele recebe as coordenadas UV interpoladas, a normal e a posição do fragmento.

Também recebe os mesmos parâmetros temporais utilizados no Vertex Shader, além dos recursos de textura.

Os principais recursos utilizados são:

- **fragUV**: coordenada UV interpolada;
- **fragNormal**: normal interpolada;
- **fragPosition**: posição do fragmento;
- **time**: tempo da animação;
- **textureImage**: imagem contendo a textura;
- **textureSampler**: sampler utilizado para realizar a leitura;
- **outColor**: cor final produzida pelo shader.

6.6 Amostragem da textura

A textura é acessada utilizando as coordenadas UV resultantes. Como o efeito dinâmico pode deslocar a coordenada para fora do intervalo $[0, 1]$, o endereçamento pode utilizar repetição.

No pseudocódigo, a operação:

$$UV_{sample} = fract(UV) \tag{6.9}$$

mantém a coordenada no intervalo $[0, 1)$ para representar o comportamento de repetição.

A leitura da textura é realizada conceitualmente por:

$$C_{texture} = Texture(UV_{sample}). \tag{6.10}$$

O Sampler define características da leitura, incluindo filtragem e endereçamento. Quando mipmaps são utilizados, o nível apropriado pode ser selecionado de acordo com a resolução aparente da textura durante a renderização.

6.7 Função procedural

Além da textura, o Fragment Shader utiliza uma função matemática para produzir uma variação procedural.

Neste pseudocódigo é utilizada uma função seno:

$$W = \sin(10U + 10V + 2t). \tag{6.11}$$

O resultado da função seno pertence ao intervalo $[-1, 1]$. Para convertê-lo para o intervalo $[0, 1]$, é aplicada a transformação:

$$W_{normalizado} = \frac{W + 1}{2}. \tag{6.12}$$

O valor resultante pode ser utilizado como intensidade para produzir uma variação visual sobre a textura.

6.8 Normalização e iluminação

A normal recebida pelo Fragment Shader é normalizada antes de participar dos cálculos de iluminação:

$$N = \frac{N}{\|N\|}. \tag{6.13}$$

Uma direção de luz também é normalizada. O fator de iluminação é obtido pelo produto escalar:

$$L = \max(0, N \cdot D), \tag{6.14}$$

em que N representa a normal normalizada e D representa a direção da luz.

O uso de max evita valores negativos quando a superfície estiver voltada para o lado oposto da fonte de iluminação.

6.9 Combinação dos resultados

O resultado procedural é convertido em uma cor:

$$C_{procedural} = (W, W, W). \tag{6.15}$$

A textura original e o resultado procedural são então combinados por meio de uma interpolação:

$$C_{mix} = (1 - a)C_{texture} + aC_{procedural}, \tag{6.16}$$

em que a representa o fator de mistura.

No pseudocódigo utilizado, $a = 0.5$, permitindo combinar igualmente os dois resultados.

6.10 Tratamento de limites

Antes de produzir a cor final, os componentes são limitados ao intervalo $[0, 1]$:

$$C_{final} = \text{clamp}(C_{mix}L, 0, 1). \tag{6.17}$$

Esse tratamento evita valores de cor fora do intervalo esperado.

O uso de `fract` nas coordenadas também impede que valores excessivamente deslocados sejam enviados diretamente para a amostragem, mantendo o comportamento de repetição da textura.

6.11 Pseudocódigo do Fragment Shader

O arquivo `uv_mapping.frag` contém o pseudocódigo correspondente ao Fragment Shader:

```
layout(location = 0) in vec2 fragUV;
layout(location = 1) in vec3 fragNormal;
layout(location = 2) in vec3 fragPosition;

layout(location = 0) out vec4 outColor;

layout(set = 0, binding = 0) uniform Parameters
{
    mat4 model;
    mat4 view;
    mat4 projection;

    float time;
    vec2 uvVelocity;
    vec2 uvScale;
} params;

layout(set = 0, binding = 1) uniform texture2D textureImage;
layout(set = 0, binding = 2) uniform sampler textureSampler;


float proceduralPattern(vec2 uv, float time)
{
    float wave =
        sin(
            uv.x * 10.0
            + uv.y * 10.0
            + time * 2.0
        );

    float normalizedWave =
        (wave + 1.0) * 0.5;

    return normalizedWave;
```

```
}


void main()
{
    vec2 uv = fragUV;

    vec2 sampledUV =
        fract(uv);

    vec4 textureColor =
        texture(
            sampler2D(textureImage, textureSampler),
            sampledUV
        );

    float proceduralValue =
        proceduralPattern(
            sampledUV,
            params.time
        );

    vec3 normal =
        normalize(fragNormal);

    vec3 lightDirection =
        normalize(vec3(0.4, 0.8, 0.6));

    float lightFactor =
        max(
            dot(normal, lightDirection),
            0.0
        );

    vec3 proceduralColor =
        vec3(
            proceduralValue,
            proceduralValue,
            proceduralValue
```

```
        );

    vec3 finalColor =
        mix(
            textureColor.rgb,
            proceduralColor,
            0.5
        );

    finalColor =
        finalColor * lightFactor;

    finalColor =
        clamp(
            finalColor,
            0.0,
            1.0
        );

    outColor =
        vec4(
            finalColor,
            textureColor.a
        );
}
```

6.12 Fluxo completo dos dados

O processamento completo pode ser resumido da seguinte forma:

Vertex Buffer → *Vertex Shader*

↓

Posição transformada + UV modificada

→ *Rasterização e Interpolação*

↓

Fragment Shader

↓

Sampling + Função Procedural + Iluminação

↓

Cor Final

→ *Color Attachment*

A aplicação fornece os dados geométricos e os parâmetros dinâmicos. O Vertex Shader transforma a geometria e prepara as coordenadas UV. A rasterização interpola esses atributos para os fragmentos. O Fragment Shader utiliza as coordenadas resultantes para acessar a textura e calcular a variação procedural.

O resultado final é enviado ao Color Attachment do Render Pass, completando o fluxo de processamento.

6.13 Relação com o UV Mapping Dinâmico e Procedural

O pseudocódigo implementa as duas características centrais estudadas neste trabalho.

No UV Mapping Dinâmico, a coordenada de textura é alterada de acordo com o tempo:

$$UV' = UV + Vt. \tag{6.18}$$

Isso permite produzir movimento contínuo da textura sem modificar a malha.

No Mapeamento Procedural, uma função matemática utiliza a coordenada UV e o tempo para produzir uma variação:

$$C = f(UV, t). \tag{6.19}$$

A combinação dos dois mecanismos permite utilizar uma textura convencional como base e, simultaneamente, adicionar uma variação calculada durante a execução do shader.

Dessa maneira, a lógica descrita nos arquivos `uv_mapping.vert` e `uv_mapping.frag` complementa a arquitetura Vulkan apresentada na Entrega 2 e descreve como o efeito pode ser implementado posteriormente em GLSL.

Capítulo 7

Aplicações práticas

7.1 Superfícies de água e materiais móveis

Uma aplicação prática do UV mapping dinâmico é a animação de superfícies que precisam transmitir sensação de movimento. Uma textura pode ser deslocada continuamente para simular o fluxo de água ou outros materiais.

A captura de uma única imagem pode mostrar a aparência da superfície, mas não demonstra completamente o movimento. Nesse caso, um vídeo de demonstração é uma evidência complementar mais adequada.

7.2 Terrenos e ambientes virtuais

Terrenos podem utilizar texturas repetíveis e procedimentos para gerar variações de aparência. Métodos procedurais também são aplicados à geração de elementos de mundos virtuais. Smelik et al. [3] discutem aplicações procedurais em terrenos, vegetação, rios, estradas, edifícios e cidades.

7.3 Materiais procedurais

Materiais como pedra, madeira, metal ou superfícies com padrões repetitivos podem ser produzidos por funções procedurais. Dong et al. [2] revisam métodos de geração procedural de texturas bidimensionais, mostrando a relevância de modelos matemáticos para produzir diferentes aparências.

7.4 Efeitos visuais

A combinação de transformação UV com tempo permite produzir efeitos de energia, lava, brilho, movimento de padrões e outros efeitos. A vantagem é que a animação pode ser obtida modificando os parâmetros de renderização, sem necessariamente alterar a geometria do objeto.

Capítulo 8

Utilização em jogos

8.1 Rocket League

O trabalho utiliza *Rocket League* como primeiro caso de análise. O jogo apresenta veículos e ambientes com materiais e texturas que contribuem para a aparência dos objetos. A documentação consultada para o trabalho inclui material da Substance 3D relacionado ao processo de modelagem e texturização de Rocket League.

No contexto deste trabalho, a técnica é relevante porque as texturas precisam ser associadas às superfícies dos veículos e elementos do cenário. O UV mapping é o mecanismo que estabelece essa correspondência entre o espaço da textura e a superfície 3D.

A análise visual deve ser complementada pela fonte técnica utilizada, pois uma captura de gameplay mostra o resultado final, mas não permite observar diretamente as coordenadas UV internas ou o shader utilizado.

8.2 Marvel's Spider-Man Remastered

O segundo caso analisado é *Marvel's Spider-Man Remastered*. O jogo utiliza modelos tridimensionais, materiais, texturas e shaders para produzir a aparência dos personagens e ambientes. O material da Insomniac Games citado na pesquisa apresenta diferenças gráficas da versão remasterizada, enquanto materiais de modificação também discutem aspectos relacionados a texturas e UVs.

Nesse caso, o UV mapping participa da associação de texturas às superfícies dos modelos, incluindo detalhes visuais das roupas e demais elementos. Assim como no primeiro caso, a captura de gameplay serve como evidência visual, enquanto as fontes

técnicas sustentam a interpretação.

8.3 Comparação

Tabela 8.1: Comparação dos casos analisados.

Jogo	**Elemento observado**	**Relação com a técnica**
Rocket League	Veículos e elementos do estádio	Associação de texturas e materiais às superfícies 3D.
Marvel's Spider-Man Remastered	Trajes, personagens e ambientes	Uso de texturas, materiais e shaders associados aos modelos.

Capítulo 9

Capturas de gameplay

9.1 Capturas de Gameplay

As capturas apresentadas nesta seção foram obtidas pelo próprio grupo durante sessões de gameplay dos jogos analisados. Elas são utilizadas como evidência visual complementar para a análise dos elementos gráficos, texturas, materiais e superfícies apresentados nos jogos.

9.2 Rocket League

As Figuras abaixo apresentam capturas próprias realizadas durante o gameplay de *Rocket League*. As imagens permitem observar diferentes elementos visuais do jogo, incluindo veículos, superfícies e elementos do ambiente.

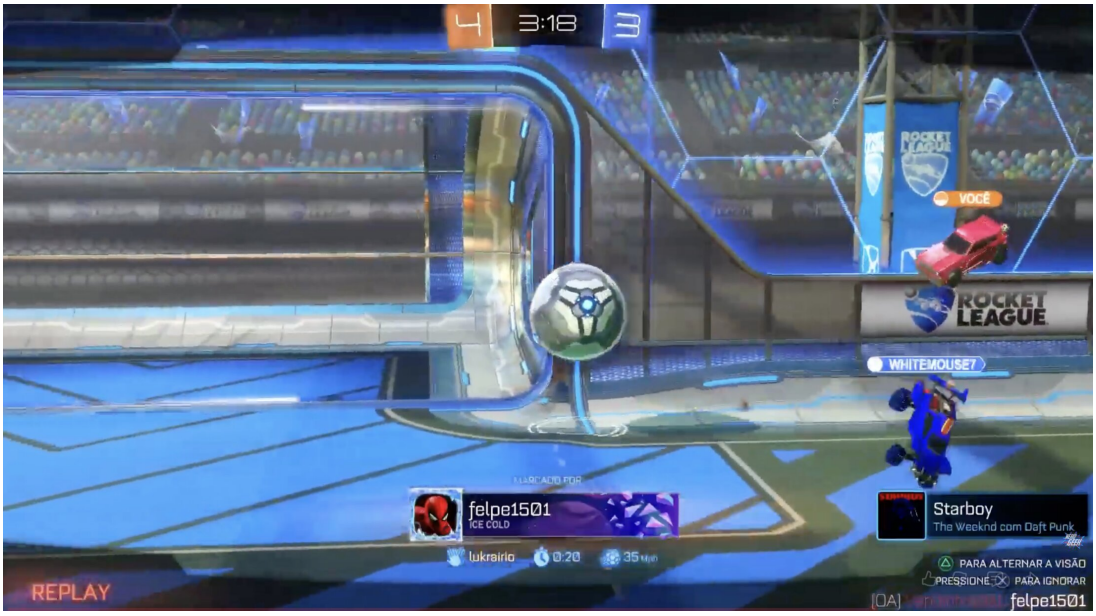


Figura 9.1: Captura própria de *Rocket League* durante o gameplay.



Figura 9.2: Segunda captura própria de *Rocket League*.

As capturas complementam a análise apresentada na seção de jogos, permitindo visualizar diretamente os elementos gráficos presentes durante a execução do jogo.

9.3 Marvel's Spider-Man Remastered

As Figuras abaixo apresentam capturas próprias realizadas durante o gameplay de *Marvel's Spider-Man Remastered*. As imagens mostram diferentes situações do jogo, permitindo observar características visuais do personagem, dos trajes e dos ambientes.



Figura 9.3: Captura própria de *Marvel's Spider-Man Remastered*.



Figura 9.4: Segunda captura própria de *Marvel's Spider-Man Remastered*.

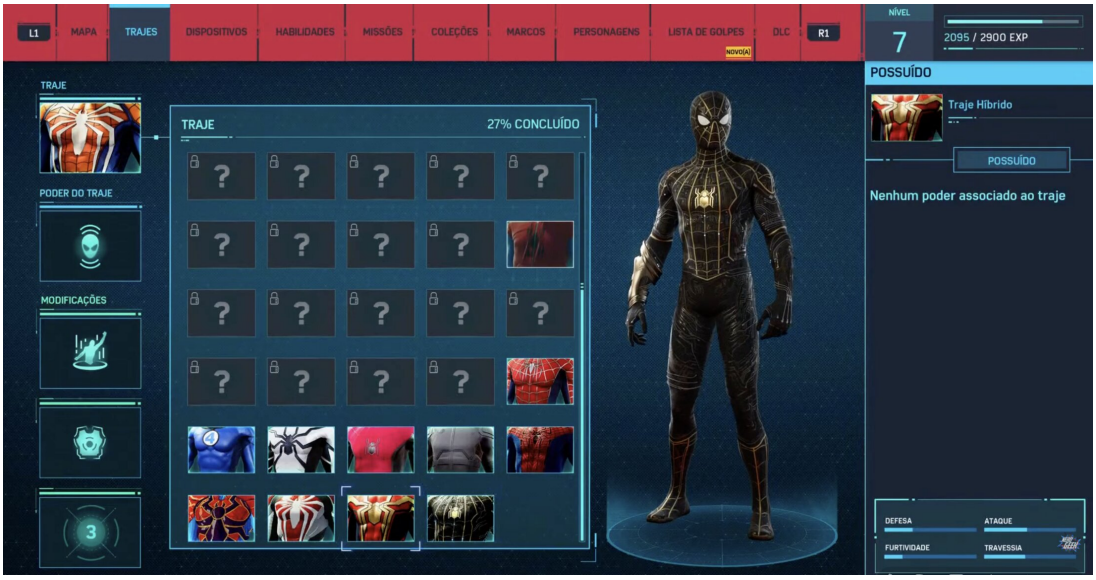


Figura 9.5: Terceira captura própria de *Marvel's Spider-Man Remastered*.

As capturas apresentadas nesta seção servem como complemento visual à análise do jogo. Embora uma imagem estática não permita observar diretamente os processos internos de aplicação das coordenadas UV, ela permite analisar o resultado visual das texturas e materiais durante o gameplay.

9.4 Limitação das Capturas

As capturas de tela permitem analisar o resultado visual apresentado pelos jogos, porém não revelam diretamente informações internas da implementação, como os valores das coordenadas UV, os shaders utilizados ou as funções responsáveis por eventuais efeitos procedurais.

Por esse motivo, as capturas são complementadas pelas demonstrações em vídeo apresentadas na seção seguinte. Os vídeos permitem observar os jogos em execução e diferentes momentos do gameplay, fornecendo uma evidência complementar às imagens estáticas.

Capítulo 10

Vídeos de demonstração

10.1 Vídeos de demonstração

Além das capturas de tela, foram utilizadas gravações de gameplay realizadas pelo grupo como material complementar para a análise dos jogos. As demonstrações foram realizadas por meio de transmissões ao vivo no canal *Nerd Geek TV*, permitindo observar os elementos apresentados durante o gameplay.

10.2 Rocket League

Para o caso de *Rocket League*, foi realizada uma transmissão ao vivo durante o gameplay. A gravação serve como evidência complementar à captura de tela, permitindo observar os veículos, o estádio, as texturas e os materiais durante a execução do jogo.

Vídeo: Gameplay de *Rocket League*
Canal: Nerd Geek TV
Tipo: transmissão ao vivo realizada pelo grupo
Link: https://www.youtube.com/live/eUX4mfkpFlQ?si=KabUl4LSs-EucEXd

10.3 Marvel's Spider-Man Remastered

Para *Marvel's Spider-Man Remastered*, foram realizadas duas transmissões ao vivo durante o gameplay. Os vídeos complementam a captura de tela e permitem observar diferentes

momentos da execução do jogo, incluindo a aparência do personagem, do traje e dos ambientes.

Vídeo 1: Gameplay de *Marvel's Spider-Man Remastered*
Canal: Nerd Geek TV
Tipo: transmissão ao vivo realizada pelo grupo
Link: https://www.youtube.com/live/mhFPk9TQWWA?si=rQEvbBQvqM9TMp5R

Vídeo 2: Gameplay de *Marvel's Spider-Man Remastered*
Canal: Nerd Geek TV
Tipo: transmissão ao vivo realizada pelo grupo
Link: https://www.youtube.com/live/RKo-mqZgxG8?si=-ZL4G0QllGjAwdU3

As gravações são utilizadas como evidência complementar às capturas de tela. Enquanto uma imagem estática permite analisar a aparência visual em um determinado momento, os vídeos permitem observar o jogo em execução e seus elementos gráficos ao longo do gameplay.

Capítulo 11

Resumo das publicações utilizadas

Esta seção atende ao requisito de apresentar um resumo de cada uma das publicações científicas ou técnicas utilizadas.

11.1 Publicação 1 – Heckbert (1986)

Referência: Heckbert, P. S. *Survey of Texture Mapping.* IEEE Computer Graphics and Applications, 1986.

O trabalho de Paul Heckbert apresenta uma visão geral dos fundamentos de texture mapping. O artigo explica que a técnica permite aumentar a riqueza visual de imagens renderizadas sem exigir que todos os detalhes sejam representados explicitamente por geometria. O autor organiza o problema em torno de dois aspectos principais: o mapeamento geométrico, que relaciona uma textura a uma superfície, e a filtragem, necessária para reduzir artefatos de aliasing.

A publicação é importante para este trabalho porque fornece a fundamentação histórica e conceitual para o uso de coordenadas de textura. O conceito de espaço de textura, representado por u e v, está diretamente relacionado ao UV mapping utilizado nas seções seguintes. A publicação também ajuda a compreender por que filtragem e amostragem são partes essenciais de uma implementação de textura.

11.2 Publicação 2 – Dong et al. (2020)

Referência: Dong, J. et al. *Survey of Procedural Methods for Two-Dimensional Texture Generation.* Sensors, 2020.

Dong et al. apresentam uma revisão dos métodos utilizados para geração procedural de texturas bidimensionais. O trabalho parte da importância das texturas para a representação de cenas realistas e discute a geração procedural como um processo baseado em modelos matemáticos.

A publicação é especialmente relevante para o tema deste trabalho porque mostra que texturas podem ser produzidas por procedimentos e funções, em vez de depender exclusivamente de imagens previamente criadas. Isso fornece base para a seção sobre mapeamento procedural e para a formulação $C = f(P, t)$, utilizada como representação simplificada de uma função que produz o resultado visual a partir de parâmetros espaciais e temporais.

11.3 Publicação 3 – Smelik et al. (2014)

Referência: Smelik, R. M.; Tutenel, T.; Bidarra, R.; Benes, B. *A Survey on Procedural Modelling for Virtual Worlds.* Computer Graphics Forum, 2014.

Smelik et al. realizam uma revisão de métodos de modelagem procedural aplicados à construção de mundos virtuais. O trabalho aborda diferentes tipos de conteúdo, incluindo terrenos, vegetação, rios, estradas, edifícios e cidades. Entre as vantagens da abordagem estão a possibilidade de gerar grandes quantidades de conteúdo e reduzir a intervenção manual.

Ao mesmo tempo, os autores destacam desafios relacionados ao controle do resultado e à interação do usuário com os métodos procedurais. Essa discussão é importante para este trabalho porque mostra que a proceduralidade não deve ser considerada apenas como uma forma de automatizar a geração: é necessário também permitir controle suficiente para que artistas e desenvolvedores obtenham o resultado desejado.

11.4 Relação entre as publicações

As três publicações complementam-se. Heckbert fornece a base para compreender o texture mapping e os problemas de mapeamento e filtragem. Dong et al. concentram-se na geração procedural de texturas bidimensionais. Smelik et al. ampliam a discussão procedural para a geração de conteúdo em mundos virtuais.

Assim, a literatura utilizada cobre desde o fundamento do mapeamento de texturas até técnicas procedurais e suas aplicações em ambientes gráficos.

Capítulo 12

Conclusão

O UV mapping é uma técnica essencial para associar informações de textura a superfícies tridimensionais. A partir do mapeamento tradicional, transformações aplicadas às coordenadas UV permitem criar comportamentos dinâmicos, enquanto métodos procedurais possibilitam calcular padrões e resultados por meio de funções matemáticas.

A revisão bibliográfica mostrou que o texture mapping possui fundamentos consolidados, incluindo problemas geométricos e de filtragem, enquanto a geração procedural amplia as possibilidades de criação e reutilização de conteúdo. As publicações analisadas também mostram que métodos procedurais podem ser úteis para produzir grandes quantidades de conteúdo, mas exigem atenção ao controle artístico e ao custo computacional.

No contexto de Vulkan, a técnica pode ser integrada ao pipeline gráfico por meio de buffers de vértices, shaders, imagens, image views e samplers. As coordenadas UV podem ser interpoladas durante a rasterização e transformadas antes da amostragem da textura, permitindo efeitos dinâmicos e procedurais.

As aplicações práticas incluem superfícies animadas, materiais procedurais, terrenos, ambientes virtuais e efeitos visuais. Os casos de Rocket League e Marvel's Spider-Man Remastered foram escolhidos para relacionar os conceitos teóricos ao resultado visual observado em jogos.

Para a versão final da entrega, as capturas de gameplay deverão ser substituídas pelos registros próprios do grupo e os vídeos deverão ser adicionados somente quando uma captura estática não for suficiente para demonstrar o comportamento analisado.

Referências Bibliográficas

[1] HECKBERT, P. S.

Survey of Texture Mapping.

IEEE Computer Graphics and Applications, v. 6, n. 11, p. 56–67, 1986.

DOI: https://doi.org/10.1109/MCG.1986.276672.

[2] DONG, J.; LIU, J.; YAO, K.; CHANTLER, M.; QI, L.; YU, H.; JIAN, M.

Survey of Procedural Methods for Two-Dimensional Texture Generation.

Sensors, v. 20, n. 4, 1135, 2020.

DOI: https://doi.org/10.3390/s20041135.

[3] SMELIK, R. M.; TUTENEL, T.; BIDARRA, R.; BENES, B.

A Survey on Procedural Modelling for Virtual Worlds.

Computer Graphics Forum, v. 33, n. 6, p. 31–50, 2014.

DOI: https://doi.org/10.1111/cgf.12276.

[4] EBERT, D. S. et al.

Texturing and Modeling: A Procedural Approach.

3. ed. San Francisco: Morgan Kaufmann, 2002.

[5] KHRONOS GROUP.

Vulkan Documentation: Image View and Sampler.

Disponível em: https://docs.vulkan.org/tutorial/latest/06_Texture_mapping/01_Image_view_and_sampler.html.

Acesso em: 2026.

[6] SUBSTANCE 3D.

Between Modeling and Texturing: Substance Designer in Rocket League.

Disponível em: https://substance3d.adobe.com/.

Acesso em: 2026.

[7] INSOMNIAC GAMES.

What is different in Marvel's Spider-Man Remastered from the original game?.

2025.

Apêndice A

Checklist da entrega

- ☐ PDF final compilado.
- ☐ Arquivos LaTeX.
- ☐ Pelo menos três publicações científicas/técnicas.
- ☐ Um resumo de cada publicação.
- ☐ Pelo menos duas aplicações práticas.
- ☐ Pelo menos dois jogos.
- ☐ Uma captura própria para cada jogo.
- ☐ Vídeo de demonstração quando a captura não for suficiente.
- ☐ Artigos consultados incluídos no ZIP.
- ☐ Diagrama da arquitetura do pipeline Vulkan.
- ☐ Arquivo-fonte do diagrama incluído no projeto.
- ☐ Explicação dos Render Passes e Subpasses.
- ☐ Descrição dos Descriptor Sets e Bindings.
- ☐ Descrição dos buffers e suas finalidades.
- ☐ Descrição dos shaders envolvidos.
- ☐ Barreiras de memória e sincronizações.
- ☐ Transições de Image Layout.
- ☐ Dependências de execução.